Appunti di ML

Alessio Novel

ML **2**
- Regole espressioni 2
- Errori 4
- Conversioni esplicite 4
- Variabili 5
- Funzioni 5
  - Patterns 6
  - Ambiente locale della funzione 7
- Print 7
- Stream 8
- Eccezioni 8
- Funzioni polimorfe 9
- Uguaglianze sui tipi 10
- Higher-order functions 11
- Curried functions 12
- Funzioni composte 12
- Tipi definiti dall'utente 12
  - Ricorsivi 13
- Signature and structures 13
- Functors 14
  - Sharings 17

ML

ML è un linguaggio di programmazione funzionale, tutto è basato sulle funzioni.
Non esistono side-effects
Le funzioni possono essere trattate come valori e come parametri di altre funzioni
Polimorfismo: si può definire uno stack che lavora sia su integer che su string (per esempio); in C questo non è possibile.
Esistono i tipi di dato astratto (strutture).
Sono preferite ricorsioni rispetto ai cicli
Tutte le variabili e le loro dimensioni sono determinabili prima di runnare il codice
Si possono dichiarare le variabili ma di solito non c'è ne bisogno

Si può avviare un programma ML in 3 modi:

- Linea di comando -> avviando poly
- Se "foo" è il nome di un file
    - poly < foo
    - use "foo";
- Standard basis

Regole espressioni

$1 + 2 * 3 = 1 + (2 * 3)$, precedenza a moltiplicazioni e divisioni
Integer:

- I negativi si scrivono con ~, per esempio ~ 1234
- Esadecimali preceduti da $0x$ oppure $\sim 0x$

Reali:

- I negativi si scrivono con ~
- Notazione esponenziale

Booleani:

- true o false (lower-case)
- Per esempio scrivere $1 = 3$ o altre equazioni equivale a dare $false$

Stringhe:

- Scritte tra doppi apici (per esempio "parola")
    - `\n`: newline
    - `\t`: tab
    - `\\`: Backslash
    - `\"`: Double-quote
    - `\xyz`: ASCII character with this code
    - `\^x`: Control character. `^G` is the same as `\007`
-
    - `\n`

Caratteri:

- Scritti cosi: #"a"

Tuple:

- Elementi eterogenei
- (1,2,3), di tipo int*int*int
  - We can mix types in tuple definitions

    ```
    val t = (4, 5.0, "six");
    val t = (4, 5.0, "six"): int * real * string
    ```

  - We can also use a complex type instead of a simple one

    ```
    > val t = (1, (2, 3, 4));
    val t = (1, (2, 3, 4)): int * (int * int * int)
    ```

-
- Per accedere ai componenti di una tupla usiamo #i dove i è l'indice (parte da 1)

List (array):

- Elementi omogenei
- $[1, 2, 3]$;
  Con hd(NomeLista) si accede al primo elemento della lista visto come unico elemento
  Con tl(NomeLista) si accede a tutto il resto della lista tranne head visto come lista
  Nel caso di una tail vuota (un solo elemento nella lista di partenza), ML determina comunque il tipo della lista (e non 'a come una empty list da zero)
  @ è l'operatore di concatenazione per le liste, ovviamente tra tipi di elementi uguali
  :: è l'operatore di per aggiungere un elemento in testa ad una lista
  nil -> empty list (nil=[])

'a:

- Tipo generico

Operazioni aritmetiche:

- +
- -
- *
- / (divisione tra reali)
- div (non DIV) (divisione tra integer)
- mod (modulo tra integer)
- 3 + 4.0 ERRORE

Operazioni stringhe:

- ^ (concatenazione)

Operazioni liste:

- @ (concatenazione)
- :: (concatenare un singolo elemento in testa ad una lista, è associativo da destra)

Operazioni di comparazione:

- =, <, >, <=, >=
- <> (not equal)
- = e <> non si possono usare per i numeri reali

Operazioni logiche:

- not
- andalso
- orelse
- Esempio: 1<2 orelse 3>4
- Logica di ottimizzazione, lazy evaluation (in un and se il primo e falso il secondo non viene guardato etc)

If-then-else:

- L'else non è opzionale
- Il tipo dopo then e dopo else deve essere lo stesso

Errori

Gli operatori in ML hanno una forma prefissa, quindi tipo $(+\ (1, 2))$, ma per certe operazioni è ammessa la forma infissa per comodità.
In questo caso la funzione + ha un solo argomento ma è una n-upla, infatti esiste il tipo n-upla
ML decide che funzione + chiamare (ne esistono molte di vario tipo) dopo aver letto il primo dato della n-upla

Conversioni esplicite

- Da integer a reale
  $real(4)$; oppure $real\ 4$;
- Da reale a integer
  - Arrotondamento (al numero più vicino, in caso di 5 si va sopra)
    $round$
  - Troncamento (elimina le cifre dopo la virgola e basta)
    $trunc$
  - Per difetto (numero più piccolo)
    $floor$
  - Per eccesso (numero più grande)
    $ceil$

    $trunc$ differisce da $floor$ perchè nei numeri negativi è come se fosse $ceil$
- Da carattere a intero
  $ord\ \#"a"$;
- Da intero a carattere
  $chr\ 97$;
- Da carattere a stringa
  $str\ \#"a"$;
- Da stringa a lista di caratteri
  $explode\ nomeStringa$;
- Da lista di caratteri a string
  $implode\ nomeLista$;

Variabili

Non serve dichiararle (ML ne deduce il tipo automaticamente)

- Tipi base
  int, real, bool, char, string
- Tipi complessi
  Per ora soltanto 2: tuple, liste

Se faccio due volte la dichiarazione di una variabile, la vecchia non smette di esistere, però non è più raggiungibile (almeno dai livelli d'ambiente alti).

Funzioni

$fun\ nomeFunzione(parametri)\ =\ \ldots;$
Quando dichiari una funzione e non dai il tipo degli argomenti, ML capisce da solo il tipo se possibile.
Per esempio
$square\ (x)\ =\ x\ *\ x;$
In automatico lo interpreta $int\ ->\ int$

Nel caso in cui volessimo noi specificare il tipo dobbiamo scrivere cosi
$square\ (x{:}\ real)\ =\ x\ *\ x;$
In realtà basta "forzare" una delle x ad essere reale e interpreterà in automatico anche le altre

Ogni funzione in ML ha esattamente un parametro, questo parametro però può essere una tupla.
Esistono i commenti con la sintassi $(*\ Questo\ \text{è}\ un\ commento\ *)$
Se si mette una variabile all'interno di una funzione, il valore che la funzione prende è quello al momento della dichiarazione, non quello al momento della chiamata

- What is the result of this code?

```
val x=3;
fun addx(a) = a+x;
val x=10;
addx(2);
```

- The value of x is the value when the function is defined

```
> addx(2);
val it = 5: int
```

Nella programmazione funzionale la ricorsione è il principale meccanismo di iterazione

Mutual recursion: due funzioni che si chiamano a vicenda in maniera ricorsiva, per fare ciò si usa and per unire le due dichiarazioni, altrimenti la chiamata della seconda funzione nella prima genera errore, non essendo ancora stato effettuata l'associazione con il nome

```
fun
   take(L) =
       if L = nil then nil
       else hd(L) :: skip(tl(L))
and
   skip(L) =
       if L = nil then nil
        else take(tl(L));

val skip = fn: ''a list -> ''a list
val take = fn: ''a list -> ''a list

> take ([1,2,3,4,5]);
val it = [1, 3, 5]: int list
```

ML capisce il tipo delle variabili analizzando i valori costanti:

- Operazione matematica
  Gli operando devono essere coerenti con l'operazione
- Comparazione
  Tutti e due gli argomenti devono essere dello stesso tipo
- Condizionale
  Gli argomenti dopo $then$ e dopo $else$ devono essere uguali
- Funzioni
  I tipi dei parametri e del valore di ritorno devono essere coerenti

Se non c'è nessun modo per determinare con certezza il tipo il tipo sarà quello di default (ad esempio integer per $+$)
Con op prima di un'operazione predefinita (tipo $+$) si può ottenere la funzione prefissa di quell'operazione.

Con il comando $open$ e dopo un tipo, possiamo vedere tutte le operazioni predefinite che si possono fare su quel tipo.

Patterns

Quando un parametro è una lista (NON VUOTA) si può scrivere $x{::}\ xs$ per avere in automatico la head in x e la tail in xs, oppure si usa $(L\ as\ x{::}\ xs)$ per avere anche la lista completa in L

Invece di usare if-then-else in una funzione ricorsiva si può usare |, e per ogni eventuale parametro che genera un'eccezione (corner-case) devo riscrivere la funzione
La lista delle alternative deve essere esaustiva

Nel caso in cui la funzione ha un parametro a cui non ci riferiamo nel corpo della funzione, si può mettere _ per farlo diventare anonymous variable

Non si può usare (n, n) come pattern per indicare che i due parametri sono uguali perchè i pattern funzionano solo per vedere che valore ha un parametro, non per confrontarli tra di loro

- I pattern non funzionano per i reali perché non possono essere comparati
- I pattern non funzionano con le concatenazioni ^ e @ perchè, ovviamente, non sanno in che modo dividere l'oggetto
- Con operazioni aritmetiche, tipo per prendere il valore $x = parametro - 1$ non possiamo scrivere il pattern $(x + 1)$

Se due alternative sono "uguali" a livello di pattern il programma darà un warning

Ambiente locale della funzione

Con la keyword let si può creare un ambiente locale dove è possibile dichiarare variabili visibili solo alla funzione (variabili locali), successivamente dopo la parola in si scrive la funzione e per chiuderla si scrive $end;$
Si può riusare lo stesso nome, il valore sarà l'ultimo associato a quel nome

```
> fun  split(nil) = (nil,nil)
     | split([a]) = ([a],nil)
     | split (a::b::cs) =
          let
            val(M,N) = split (cs)
          in
            (a::M,b::N)
          end;
val split = fn: 'a list -> 'a list * 'a list
```

Print

$print$ è una funzione che prende come argomento una stringa e ritorna void (unit) quindi $string \rightarrow unit$. Il confronto fra $unit$ da sempre true.
Per convertire da numero reale/numero intero/boolean a stringa esistono delle funzioni primitive $Real.toString\ Int.toString\ Bool.toString$.
Per i caratteri abbiamo già visto che esiste $str(c)$

Si possono scrivere più funzioni alla volta, in questo caso il valore di ritorno sarà quello dell'ultima funzione chiamata, con questa sintassi:
$(print("a"); equalReal(2.0, 3.1));$

Stream

Si può creare una variabile di stream in input con la sintassi:
$val\ infile = TextIO.openIn\ ("test");$
Dove $test$ è il nome del file
Ci sono alcune funzioni utili per gli stream:

- $TextIO.endOfStream\ (infile)$
  Ritorna un bool in base a sé lo stream è arrivato alla fine o meno
- $TextIO.inputN\ (infile, 4)$
  Ritorna una stringa lunga 4 caratteri, quando si arriva a fine file ritorna stringhe vuote
- $TextIO.inputLine\ (infile)$
  Ritorna tutta la linea in formato string option
  - $SOME$ "..." se non è arrivato a fine file
  - $NONE$ se è arrivato a fine file
- $TextIO.input\ (infile)$
  Ritorna tutto lo stream fino alla fine, ovviamente in base a dove inizia in quel momento, alle chiamate successive ritorna stringhe vuote
- $TextIO.input1(infile)$
  Ritorna un carattere in formato char option
  - $SOME$ "..." se non è arrivato a fine file
  - $NONE$ se è arrivato a fine file
- $TextIO.lookahead(infile)$
  Ritorna un carattere in formato char option ma senza scorrere lo stream
- $TextIO.canInput(infile, n)$
  Ritorna un int option con il numero di caratteri contenuti (ci sono almeno n caratteri?), se il numero di caratteri è maggiore o uguale a n ritorna n, altrimenti ritorna il numero di caratteri rimasti
- $TextIO.closeIn(infile);$
  Per chiudere lo stream

Eccezioni

L'utente può dichiarare delle eccezioni con $exception\ nome;$ e con $raise\ nome$ si solleva l'eccezione in una funzione.
Per dichiarare eccezioni con parametri si usa $exception\ nome\ of\ tipo1\ *\ tipo2\ *\ ...;$
Successivamente dobbiamo fare una funzione che farà da $handle$ per le eccezioni della funzione che definiamo come da tenere sotto controllo, e lì dentro metteremo le eccezioni.

Quando vogliamo chiamare la funzione, dobbiamo chiamare l'handle, che eseguirà la funzione e gestirà gli errori (quelli conosciuti) dentro ad handle.
Quando gestisco le eccezioni devo usare la sintassi $nomeEccezione(params) \ => \ qualcosa$

```
exception OutOfRange of int * int;

fun comb1(n,m)=
   if n <= 0 then raise OutOfRange (n,m)
   else if m<0 orelse m>n then raise OutOfRange (n,m)
   else if m=0 orelse m=n then 1
   else comb1 (n-1,m) + comb1 (n-1,m-1);
val comb1 = fn: int * int -> int

fun comb (n,m) = comb1 (n,m) handle
   OutOfRange (0,0) => 1 |
   OutOfRange (n,m) => (
         print ("out of range: n=");
         print (Int.toString(n));
         print (" m=");
         print (Int.toString(m));
         print ("\n");
         0
     );
val comb = fn: int * int -> int
```

Funzioni polimorfe

ML è sicuro sui tipi, infatti è possibile determinare ogni tipo di un programma prima di avviarlo. Se abbiamo una funzione di tipo $'a$ (cioè qualsiasi) possiamo richiamarla nello stesso comando (stessa linea) con parametri di tipo diverso senza problemi.

Per questo motivo esistono funzioni predefinite di ml che hanno tipo $'a$ e possono essere chiamate su un qualsiasi tipo, ad esempio la reversione di una lista con rev

```
> fun f(x) =
    if x<10 then identity
    else rev;
val f = fn: int -> 'a list -> 'a list
```

La funzione f sceglierà in base al valore di x quale funzione applicare a una lista, che dovrà essere passata come parametro del parametro: $f\ 3\ [1, 2, 3]$

Uguaglianze sui tipi

Le uguaglianze con i simboli $=$(uguale) e $<>$(diverso) sono permesse solo sui seguenti tipi (oppure tuple o liste dei seguenti tipi):
Integer, booleans, characters
Questi tipi sono indicati con $''a$
Ma non sono permesse sui reali.

Non si possono usare le uguaglianza direttamente sulle funzioni.

Usando gli if non si possono confrontare liste di tipi che non sono uguagliabili (ad esempio liste di reali o liste di funzioni), però possiamo ovviare a ciò usando i pattern.

- Reverse a list

```
> fun rev1 (L) =
     if L = nil then nil
     else rev1(tl(L)) @ [hd(L)];
val rev1 = fn: ''a list -> ''a list
```

- But this only accepts lists of equality type
- The reason is the test `L=nil`
- We can avoid this as follows

```
>  fun rev2 (nil) = nil
    |  rev2(x::xs) = rev2 (xs) @ [x];
val rev2 = fn: 'a list -> 'a list
```

Un altro modo per vedere se una lista di tipi non uguagliabili è vuota è usare la funzione $null(list)$

Higher-order functions

Funzioni che prendono funzioni come argomenti
Ad esempio la funzione map (built-in), prende una funzione e una lista e applica ad ogni elemento la funzione ritornando una lista dei risultati.

```
> fun simpleMap (F,nil) = nil
    |  simpleMap (F,x::xs) = F(x) :: simpleMap(F,xs);
val simpleMap = fn: ('a -> 'b) * 'a list -> 'b list
```

A queste funzioni possiamo passare anche funzioni anonime, ovvero definite come $fn\ x\ =>$ ...
Invece la funzione $reduce$ prende una funzione e una lista e applica la funzione tra il primo e il secondo, poi tra il risultato e il terzo e così via.
Si possono usare come funzioni anche operatori unari (come ~), oppure funzioni anonime, cioè definite direttamente durante la chiamata della funzione principale.

- Using a unary operator

```
> simpleMap (~, [1,2,3]);
val it = [~1, ~2, ~3]: int list
```

- Using an anonymous function

```
> simpleMap ( fn x => x*x, [1.0,2.0,3.0]);
val it = [1.0, 4.0, 9.0]: real list
```

```
> exception EmptyList;
exception EmptyList
```

```
> fun reduce (F,nil) = raise EmptyList
    | reduce (F,[a]) = a
    | reduce (F,x::xs) = F (x, reduce (F,xs));
val reduce = fn: ('a * 'a -> 'a) * 'a list -> 'a
```

Con la keyword op prima di un operatore infisso, possiamo ottenere l'operatore prefisso per quella funzione, cosi possiamo usarlo per queste operazioni sulle liste.

Resoconto funzioni built-in:

- map
  Applica la funzione ad ogni elemento della lista
- reduce
  Applica la funzione a due elementi della lista alla volta, partendo dalla coda, per partire dalla testa bisogna usare $lreduce$
- filter
  Crea una lista con solo gli elementi che rispettano una condizione booleana

- tabulate
  Applica la funzione ad n elementi partendo da a e sommando ogni volta $delta$
- applyList
  Applica ogni funzione contenuta in una lista di funzioni ad un elemento e genera una lista coi risultati

Curried functions

Le funzioni in ML al massimo possono prendere in input delle t-uple.
Si possono dichiarare delle funzioni che accettano più parametri come input, e quindi ad esempio una funzione con due parametri x e y, avrà come risultato una funzione costante rispetto ad x che poi verrà applicata alla y.
Questo è comodo per usare delle funzioni "parziali", ovvero dove fissiamo un solo parametro.

Dobbiamo stare attenti all'ordine con cui vengono letti i parametri, infatti scrivendo $fun\ f\ c{:}\ char$ ML capirà $fun\ (f\ c){:}\ char$, quando probabilmente volevamo intendere $fun\ f\ (c{:}\ char)$

Funzioni composte

La composizione della funzione $y = F(x)$ e $z = G(y)$ è la funzione $z = H(x) = G(F(x))$
In ml l'operatore o consente di fare funzioni composte.
Esistono delle funzioni builtin che sono $foldr$ e $foldl$ che applicano una funzione a 2 parametri agli elementi di una lista, rispettivamente da destra o da sinistra.

Tipi definiti dall'utente

Scrivendo $type\ nomeTipo\ =$ possiamo definire delle abbreviazioni per un tipo. (oppure $eqtype$)
Possiamo usare dei parametri di input per il tipo, ad esempio usando 'a eccetera, possiamo creare un tipo di tipo indefinito.
Questa però è solo una dichiarazione di un tipo, e quando si farà il confronto con un tipo avente la stessa struttura verrà eseguito il confronto normale in base ai tipi base.

Invece con $datatype$ creiamo proprio dei nuovi tipi.
Bisogna definire:
- Il costruttore del tipo: il nome del tipo
- I costruttori dei dati: i valori che può assumere

Ad esempio $datatype\ fruit\ =\ Apple \mid Pear \mid Grape$;
In questa situazione $Apple, etc$ sono dette constructor expression, e possiamo anche parametrizzarli scrivendo $Banana\ of\ int$

Si possono anche inserire più costruttori con diverso numero (e/o tipo) di parametri (unioni)

```
>  datatype ('a,'b) element =
        P of 'a * 'b |
        S of 'a;
datatype ('a, 'b) element = P of 'a * 'b | S of 'a

> P ("a",1);
val it = P ("a", 1): (string, int) element

> P(1.0,2.0);
val it = P (1.0, 2.0): (real, real) element

> S(["a","b"]);
val it = S ["a", "b"]: (string list, 'a) element
```

Questo è molto utile quando abbiamo dei dati “diversi” su cui dobbiamo fare le stesse operazioni

Possiamo ad esempio usare un tipo per fare un albero binario, che altrimenti sarebbe impossibile visto che i figli di un nodo devono essere o alberi o elementi vuoti allo stesso tempo.

```
> datatype 'label btree =
    Empty  |
    Node of 'label * 'label btree * 'label btree;
datatype 'a btree = Empty | Node of 'a * 'a btree * 'a btree
```

Ricorsivi

Si possono anche definire tipi mutuamente ricorsivi con la sintassi and (come per le funzioni)

Signature and structures

Le structure sono delle classi statiche con delle funzioni.
Le signature sono delle interfacce in cui si definiscono le firme delle funzioni, e poi verranno implementate in una struct usando : >
Quando una structure “implementa” una signature, se definiamo nuove funzioni che non sono presenti nella signature, non saranno visibili all’esterno della struct.
Volendo si può creare una struttura e successivamente creare una signature compatibile che restringe in qualche modo la struttura e creare una nuova struttura ristretta.

Se proviamo ad assegnare un elemento della struttura ad una variabile che strutturalmente è dello stesso tipo, viene dato un errore perché non possiamo sapere come è implementato quel dato dentro la struttura
Con “open” si dichiarano tutte i metodi interni alla struttura come “liberi”, e quindi utilizzabili senza riferirsi alla struttura, evitando di riscrivere ogni volta il nome della struttura.

```
structure Mapping = struct
 exception NotFound;
 val create = nil;
 fun lookup (d,nil) = raise NotFound
   | lookup (d,(e,r)::es) =
        if d=e then r
        else lookup (d,es);

 fun insert (d,r,nil) = [(d,r)]
   | insert (d,r,(e,s)::es) =
       if d=e then (d,r)::es
       else (e,s)::insert(d,r,es)
end;
```

```
signature SIMAPPING = sig
  val create : (string * int) list;
  val insert : string * int * (string * int) list -> (string * int) list;
  val lookup : string * (string * int) list -> int
end;
```

```
> structure SiMapping : SIMAPPING = Mapping;

structure SiMapping: SIMAPPING
```

Functors

È un oggetto che prende in input una (o più) strutture e ne da in output un’altra “modificata”, ad esempio specificando i tipi lasciati indefiniti.

The steps we take

- Define a signature `TOTALORDER` that is satisfied by our functor inputs
- Define a functor `MakeBST` that takes a structure S with signature `TOTALORDER` and produces a structure
- Define structure `STRING` with signature `TOTALORDER` and with a comparison operator on strings
- Apply `MakeBST` to `String` to produce the desired structure

Step 1: `TOTALORDER`

```
> signature TOTALORDER = sig
   type element;
   val lt : element * element -> bool
end;

signature TOTALORDER = sig type element val lt: element * element ->
bool end
```

Step 2: Defining the functor (sketch)

```
>  functor MakeBST (Lt: TOTALORDER):
     sig
        type 'label btree
        exception EmptyTeee;
        val create : Lt.element btree;
        val lookup : Lt.element * Lt.element  btree -> bool;
        val insert : Lt.element * Lt.element  btree -> Lt.element bt
        val deletemin : Lt.element btree -> Lt.element Lt.element bt
        val delete : Lt.element  * LT.element btree -> Lt.element bt
    end
=
   struct
      open Lt;
      datatype 'label btree =
        Empty |
        Node of 'label * 'label btree * 'label btree;
      val create = Empty;
      val lookup (x, Empty) = ...
      val insert (x, Empty) = ...
      exception EmptyTree;
      fun deletemin (Empty) = ...
      fun delete (x,Empty) = ...
end:
```

3. Functor argument

```
structure String: TOTALORDER =
struct
    type element = string;
    fun lt (x,y)  =
      let
        fun lower (nil) = nil
        |   lower (c::cs) = (Char.toLower c)::lower (cs);
      in
        implode (lower (explode (x))) < implode (lower( explode (y)))
      end;
end;
```

4. Applying the functor

```
structure StringBST = MakeBST (String);
```

Sharings

Obbliga alcune parti di una signature ad essere uguali